

- Description

This module manages a smart warehouse environment monitoring system, ensuring that:

1. **The temperature and humidity inside the warehouse are continuously monitored to maintain optimal storage conditions.**
2. **If the temperature exceeds the upper limit, the cooling system is activated to prevent overheating.**
3. **If the humidity goes beyond safe levels, ventilation or dehumidification is triggered.**
4. **A log of all temperature and humidity readings, system activations, and environmental optimizations is maintained for regulatory compliance.**

This system helps protect stored goods, optimize warehouse conditions, and improve energy efficiency.

- Specification

module SmartWarehouseEnvironmentMonitoring

var currentTemperature, maxTemperature, minTemperature, currentHumidity, maxHumidity, minHumidity: real

var coolingActive, dehumidifierActive: boolean

var log: sequence of string

process updateEnvironment(tempChange: real, humidityChange: real)

functional scenario form:

currentTemperature <= maxTemperature && Environment conditions updated successfully.

||

currentTemperature > maxTemperature && coolingActive = true, log = log $\oplus$ ("Overheating Warning: " + currentTemperature + "°C, Cooling System Activated."), Overheating warning triggered, cooling system activated.

||

currentTemperature < minTemperature && log = log $\oplus$ ("Low Temperature Alert: " + currentTemperature + "°C, Heating System Activated."), Low temperature alert triggered, heating system activated.

||

currentHumidity > maxHumidity && dehumidifierActive = true, log = log $\oplus$ ("High Humidity Warning: " + currentHumidity + "%, Dehumidifier Activated."), High humidity warning triggered, dehumidifier activated.

||

currentHumidity < minHumidity && log = log $\oplus$ ("Low Humidity Alert: " + currentHumidity + "%, Humidifier Activated."), Low humidity alert triggered, humidifier activated.

end process

end module

- Java Code

```

package code;
import java.util.*;
class WarehouseEnvironment {
    private double maxTemperature;
    private double minTemperature;
    private double maxHumidity;
    private double minHumidity;
    private double currentTemperature;
    private double currentHumidity;
    private boolean coolingActive;
    private boolean dehumidifierActive;
    private List<String> log;
    public WarehouseEnvironment(double maxTemperature, double minTemperature, double maxHumidity, double minHumidity, double initialTemperature, double initialHumidity) {
        this.maxTemperature = maxTemperature;
        this.minTemperature = minTemperature;
        this.maxHumidity = maxHumidity;
        this.minHumidity = minHumidity;
        this.currentTemperature = initialTemperature;
        this.currentHumidity = initialHumidity;
        this.coolingActive = false;
        this.dehumidifierActive = false;
        this.log = new ArrayList<>();
    }
    public void updateEnvironment(double tempChange, double humidityChange) {
        currentTemperature += tempChange;
        currentHumidity += humidityChange;
        manageClimateControl();
        String message = String.format("Current Temp: %.2f°C | Humidity: %.2f%", currentTemperature, currentHumidity);
        if (currentTemperature > maxTemperature) {
            message += " | WARNING: Overheating! Cooling System Activated.";
        } else if (currentTemperature < minTemperature) {
            message += " | ALERT: Too Cold! Adjusting Heating System.";
        }
        if (currentHumidity > maxHumidity) {
            message += " | WARNING: High Humidity! Dehumidifier Activated.";
        } else if (currentHumidity < minHumidity) {
            message += " | ALERT: Low Humidity! Adjusting Humidification System.";
        }
    }
}

```

```

    }
    System.out.println(message);
    log.add(message);
}
private void manageClimateControl() {
    coolingActive = currentTemperature > maxTemperature;
    dehumidifierActive = currentHumidity > maxHumidity;
}
public void printLog() {
    System.out.println("\nWarehouse Environment Log:");
    for (String logEntry : log) {
        System.out.println(logEntry);
    }
}
}

public class c51_SmartWarehouseSystem {
    public static void main(String[] args) {
        WarehouseEnvironment warehouse = new WarehouseEnvironment(30.0,
        5.0, 70.0, 30.0, 20.0, 50.0);
        // testcase-VT:
        //     warehouse.updateEnvironment(8.0, 10.0);
        //     warehouse.updateEnvironment(5.0, 20.0);
        //     warehouse.updateEnvironment(-10.0, -15.0);
        //     warehouse.updateEnvironment(-20.0, -30.0);
        //     warehouse.updateEnvironment(-20.0, -30.0);
        //     warehouse.updateEnvironment(-20.0, -70.0);
        //     warehouse.printLog();

        // testcase-FT:
        //     warehouse.updateEnvironment(0.0, 0.0);
        //     warehouse.updateEnvironment(-1.0582717256057185E308, 0.0);
        //     warehouse.updateEnvironment(-1.7836486571955191E308, 0.0);
        //     warehouse.updateEnvironment(0.0, -1.0582717256057185E308);
        //     warehouse.updateEnvironment(0.0, -1.7836486571955191E308);
        //     warehouse.updateEnvironment(1.5448925437890742E307, -
        1.797654840028587E308);
        //     warehouse.printLog();

        // testcase-Z3:
        //     warehouse.updateEnvironment(8, 10);
        //     warehouse.updateEnvironment(5, 20);
        //     warehouse.updateEnvironment(-10, -15);
        //     warehouse.updateEnvironment(-20, -30);
        //     warehouse.updateEnvironment(-20, -30);
    }
}

```

```
//      warehouse.updateEnvironment(-20, -70);
//      warehouse.printLog();

// testcase-UVT:
warehouse.updateEnvironment(8.0, 10.0);
warehouse.updateEnvironment(5.0, 20.0);
warehouse.updateEnvironment(-10.0, -15.0);
warehouse.updateEnvironment(-20.0, -30.0);
warehouse.updateEnvironment(-20.0, -30.0);
warehouse.updateEnvironment(-20.0, -70.0);
warehouse.printLog();
}
}
```

- Coverage report

1. Test case number: 6
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.7 %	14	828	842
✓ src	1.7 %	14	828	842
✓ code	1.7 %	14	828	842
✓ c51_SmartWarehouseSystem.java	100.0 %	14	0	14
> c51_SmartWarehouseSystem		0	0	0
> WarehouseEnvironment	100.0 %	14	0	14

3. Vibration testing instruction coverage: 98.4%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.9 %	189	19,975	20,164
✓ src	0.9 %	189	19,975	20,164
✓ code	0.9 %	189	19,975	20,164
✓ c51_SmartWarehouseSystem.java	98.4 %	189	3	192
> c51_SmartWarehouseSystem	92.5 %	37	3	40
> WarehouseEnvironment	100.0 %	152	0	152

4. Fuzz testing branch coverage: 71.4%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.2 %	10	832	842
✓ src	1.2 %	10	832	842
✓ code	1.2 %	10	832	842
✓ c51_SmartWarehouseSystem.java	71.4 %	10	4	14
> c51_SmartWarehouseSystem		0	0	0
> WarehouseEnvironment	71.4 %	10	4	14

5. Fuzz testing instruction coverage: 92.2%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.9 %	177	19,987	20,164
✓ src	0.9 %	177	19,987	20,164
✓ code	0.9 %	177	19,987	20,164
✓ c51_SmartWarehouseSystem.java	92.2 %	177	15	192
> c51_SmartWarehouseSystem	92.5 %	37	3	40
> WarehouseEnvironment	92.1 %	140	12	152

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.7 %	14	828	842
▼ src	1.7 %	14	828	842
▼ code	1.7 %	14	828	842
▼ c51_SmartWarehouseSystem.java	100.0 %	14	0	14
> c51_SmartWarehouseSystem		0	0	0
> WarehouseEnvironment	100.0 %	14	0	14

7. Z3 instruction coverage: 98.4%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.9 %	189	19,975	20,164
▼ src	0.9 %	189	19,975	20,164
▼ code	0.9 %	189	19,975	20,164
▼ c51_SmartWarehouseSystem.java	98.4 %	189	3	192
> c51_SmartWarehouseSystem	92.5 %	37	3	40
> WarehouseEnvironment	100.0 %	152	0	152

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.7 %	14	828	842
▼ src	1.7 %	14	828	842
▼ code	1.7 %	14	828	842
▼ c51_SmartWarehouseSystem.java	100.0 %	14	0	14
> c51_SmartWarehouseSystem		0	0	0
> WarehouseEnvironment	100.0 %	14	0	14

9. Unimproved V-method instruction coverage: 98.4%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.9 %	189	19,975	20,164
▼ src	0.9 %	189	19,975	20,164
▼ code	0.9 %	189	19,975	20,164
▼ c51_SmartWarehouseSystem.java	98.4 %	189	3	192
> c51_SmartWarehouseSystem	92.5 %	37	3	40
> WarehouseEnvironment	100.0 %	152	0	152